Ders 5 Çalışma Özeti

Ders 5 Çalışma Özeti

Genel Konular

- Process (İşlem) Kavramı
  - Process = programın çalışan hali. Diskte duran program pasif, çalışmaya başladığında process oluşur.
  - Bir program birden fazla process oluşturabilir (tersi mümkün değil).
  - Process'in çalışabilmesi için: programın belleğe yüklenmesi, çalışma anında eriştiği kaynaklar, çalıştıracağı bir sonraki komutun yeri, ürettiği veri vb. bilgilerin tutulması gerekir.
  - Tarihsel olarak: task, job, process terimleri birbirinin yerine kullanılmıştır (interchangeable).
- Process'in Bileşenleri
  - Text section: Programın kodu (instruction'lar).
  - Program Counter: Hangi instruction'da olduğumuz.
  - Stack: Fonksiyon çağrı/return, lokal değişkenler, interrupt'tan dönüş adresleri.
  - Data section: Global değişkenler.
  - Heap: Dinamik bellek tahsisi (malloc/new ile).
- Process Memory Organizasyonu
  - Her process'e tüm bellek kendisine aitmiş gibi gösterilir (sanal bellek).
  - Text (kod), data (global değişkenler), stack (yukarı doğru büyür), heap (aşağı doğru büyür) birbirine doğru genişler.
- Process State (İşlem Durumu)
  - New: Process oluşturuluyor.
  - Ready: CPU atanmasını bekliyor.
  - Running: İşlemci tarafından çalıştırılıyor.
  - Waiting: Bir olayın gerçekleşmesini bekliyor (I/O tamamlanması, event vb.).
  - Terminated: İşini bitirdi, sonlanıyor.
  - "Terminated" durumu önemli: Process hemen sistemden çıkmaz, parent process exit kodunu okuyabilmelidir.
- Process Control Block (PCB)
  - İşletim sisteminin her process için tuttuğu veri yapısı ("process'in sicili").
  - İçeriği: Process state, Program Counter, CPU register'ları, Scheduling bilgisi, Memory management bilgisi, Accounting bilgisi, I/O durumu, Açık dosyaların listesi.
  - Linux'ta bu yapıya "task_struct" adı verilir.
- Scheduling ve Context Switch
  - Scheduling: Ready queue'dan bir process'in seçilip CPU'ya atanması.
  - Context Switch: Bir process'in CPU'dan alınıp diğerinin verilmesi. PCB güncellenir (register'lar, program counter saklanır/geri yüklenir).
  - Context switch sırasında arada boşluk (idle) oluşur; bu süre az olmalıdır.
  - Context switch'i hızlandırmak için bazı işlemcilerde birden fazla register seti bulunur (ör. Sun işlemcilerinde 32 register seti). Pointer değiştirilerek hızlıca geçiş yapılabilir.
- Kuyruk Yapıları
  - Ready queue: CPU atanmasını bekleyen process'ler.
  - I/O device queues: Her cihaz için ayrı kuyruk.
  - Parent-child ilişkisi: Process'ler ağaç yapısında organize edilir.
  - Listeler genellikle linked list olarak implemente edilir.

Hocanın Özellikle Vurguladığı Kısımlar

- Process'in Yaşam Döngüsünde Terminate Durumu
  - Hoca özellikle vurgular: "Niye öyle bir state var diye düşünebilirsiniz." Process işini bitirdiğinde hemen silinmez; çünkü parent process'in exit kodunu okuması gerekir. Eğer terminate state olmasaydı, child'ın sonucu okunamadan kaybolurdu.
- Program vs Process Ayrımı
  - Hoca "Program pasif, işlem aktif" vurgusunu yapar. Nesne yönelimli programlamadan örnek verir: Sınıfınız (class) = program; oluşturduğunuz objeler = process'ler.
- Context Switch Performansı
  - Hoca, context switch zamanının mümkün olduğunca düşük olması gerektiğini vurgular. Sürekli 0-1 arasında geçiş yapılırsa, gerçek işler için zaman kalmaz. Birden fazla register seti gibi donanımsal çözümler olsa da sınırsız değildir.
- Stack ve Register Yapısı
  - Hoca, register'ların "anlık erişilen fiziksel register'lar" olduğunu, ancak bir process durdurulup başka process'e geçileceği zaman bu değerlerin kenara saklanması gerektiğini vurgular. Bu saklama alanı aslında PCB içindeki register alanıdır.

Kısa Tekrar Notları

- Process = programın çalışma hali.
- Process bileşenleri: text + PC + stack + data + heap.
- Process state'leri: New → Ready → Running → Waiting → Terminated.
- PCB içeriği: state, PC, register, scheduling, memory mgmt, accounting, I/O, dosya listesi.
- Context switch: PCB güncellemesi, register saklama/yükleme.
- Ready queue + I/O queues = toplam scheduling altyapısı.
- Linux task_struct = PCB.
- File descriptor: 0 (stdin), 1 (stdout), 2 (stderr).

Detaylı Açıklamalar

Ders 5, process (işlem) kavramını derinlemesine ele alır. Bu ders, geçen haftalarda "program" kavramından "process" kavramına geçişi somutlaştırır. Process = programın çalışan hali; diskte duran program pasif, çalışmaya başladığında aktif hale geçer.

Process'in yapısal bileşenleri detaylı şekilde açıklanır: Text section (programın kodu, normalde değişmez), Program Counter (hangi instruction'dayız), Stack (fonksiyon çağrı/return adresleri, lokal değişkenler, interrupt'tan dönüş), Data section (global değişkenler), Heap (dinamik bellek tahsisi). Stack yukarı doğru büyür, heap aşağı doğru büyür; birbirlerine doğru genişler.

Process state'leri (yeni, hazır, çalışıyor, bekliyor, sonlanmış) detaylı şekilde anlatılır. State'ler arası geçişler, scheduler'ın hangi durumda devreye girdiği, I/O işlemi başlatan process'in waiting state'e geçmesi gibi detaylar tartışılır. "Terminate" durumunun neden var olduğu özellikle açıklanır: Parent process, child'ın çıkış kodunu (exit code) okuyabilmelidir. Bu olmadan çocuk process'in ürettiği sonuç kaybolur.

Process Control Block (PCB), OS'nin her process için tuttuğu veri yapısıdır. İçinde process'in tüm durum bilgisi (program counter, register değerleri, açık dosyalar, scheduling bilgisi, accounting bilgisi vb.) yer alır. Linux'ta bu yapı "task_struct" adını alır ve süreçler arası bağlantıyı (parent-child) linked list şeklinde tutar. Default olarak her process'in 3 file descriptor'ı vardır: 0 (stdin), 1 (stdout), 2 (stderr).

Scheduling ve Context Switch kavramları detaylı anlatılır. Scheduling, ready queue'dan uygun process'i seçip CPU'ya atama işlemidir. Context switch ise bir process'in CPU'dan alınıp diğerine geçilmesi sürecidir. Bu sırada register değerleri, program counter saklanır ve yeni process için yüklenir. Context switch süresi verimliliği doğrudan etkiler; çok sık context switch yapılması "thrashing" etkisi yaratır. Çözüm olarak bazı işlemciler birden fazla register seti içerir (Sun işlemcilerinde 32 set), pointer değiştirilerek hızlı geçiş sağlanır.

Hoca, process'ler arası parent-child ilişkisini ağaç yapısı şeklinde açıklar. İlk process (init/systemd) kök process'tir; tüm diğer process'ler bu yapıdan türetilir. Bu ilişki sayesinde çocuk process'ler sonlandığında parent'a haber verilir.

Kuyruk yapıları OS'nin temel veri yapılarından biridir. Ready queue, I/O device queues (her cihaz için ayrı) linked list olarak implemente edilir. Process'ler kuyruklara eklenir, uygun koşul sağlandığında kuyruktan çıkarılır. Multi-core sistemlerde her core için ayrı ready queue olabilir; bu durum senkronizasyon problemlerini beraberinde getirir.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.